Soutenance Projet 7 OpenClassrooms

Les petits plats

Développez un algorithme de recherche en JavaScript

Présenter par Ghani

PROJET 7

Les petits plats

01

Présentation

02

Les factories

03

Les utils

Présentation

L'entreprise, spécialisée dans les livres de cuisine, se lance dans la création d'un site de recettes. Sandra, en tant que cheffe de projet, orchestre ce projet avec des freelances. L'objectif initial est d'améliorer la performance du moteur de recherche du site, basé sur un fichier JavaScript contenant 50 recettes. La maquette visuelle est définie sur Figma, et l'accent est mis sur une recherche rapide.

Jean-Baptiste, le Lead Developer, préconise deux implémentations distinctes de la fonction de recherche pour les comparer. Une "fiche d'investigation de fonctionnalité" documentera ces implémentations, dont l'une utilisera des boucles natives et l'autre des méthodes Array pour assurer la performance.

La dernière étape implique la comparaison des performances de ces deux implémentations à l'aide de Jsben.ch, permettant de déterminer la plus performante. Cette recommandation guidera le choix final. L'interface graphique peut être initialement développée avec Bootstrap ou Tailwind CSS, en suivant les bonnes pratiques et en garantissant la sécurité. Les deux implémentations seront ensuite développées en parallèle sur des branches Git distinctes.

En somme, le projet vise à créer un moteur de recherche performant pour un site de recettes, en développant et comparant deux implémentations de recherche pour déterminer la meilleure approche.

FACTORIES

recipes.js

Ce code JavaScript est une partie essentielle de l’application de recettes en ligne. Il comporte trois fonctions principales :

1. `displayGallery(filteredRecipes)` : Cette fonction affiche les recettes filtrées dans l'interface utilisateur en créant dynamiquement des cartes de recettes à partir des données fournies.

2. `createCard(recipe)` : Elle génère la structure HTML de chaque carte de recette en utilisant des données spécifiques de la recette, comme l'ID, la description, les ingrédients et le temps de préparation.

3. `recipesCounter()` : Cette fonction met à jour le compteur de recettes et gère les messages d'erreur en fonction du nombre de recettes filtrées.

L'ensemble de ces fonctions contribue à offrir une expérience utilisateur fluide lors de la recherche et de la visualisation des recettes sur le site web.

recipes.js

```javascript
function displayGallery(filteredRecipes) {
  const section = document.getElementById("recipes");
  section.innerHTML = "";

  // Parcourir les recettes filtrées et afficher chaque carte de recette
  filteredRecipes.forEach((recipe) => {
    const { id } = recipe;
    const cardModel = createCard(recipe);

    // Créer un nouvel élément d'article pour la carte de recette
    const cardEl = document.createElement("article");
    cardEl.classList.add("recipe_card");
    cardEl.setAttribute("article-ID", `${id}`);
    cardEl.innerHTML = cardModel;

    section.appendChild(cardEl);
  });
}

function createCard(recipe) {
  const { id, description, ingredients, name, time } = recipe;

  let ingredientItems = "";
  for (let i = 0; i < ingredients.length; i++) {
    const ingredient = ingredients[i];        ghanigh, le mois dernier • searchbar
    const ingredientItem = ingredient.ingredient;
    const quantity = ingredient.quantity ? `${ingredient.quantity}` : " ";
    const unit = ingredient.unit ? `${ingredient.unit}` : " ";

    // Remplacer l'unité "grammes" par "g" pour une meilleure lisibilité
    if (unit === "grammes") {
      ingredient.unit = "g";
    }
```

```javascript
  // Retourner la carte de recette
  return card;
}

function recipesCounter() {
  const counterWrapper = document.querySelector(".recipes_counter");
  const count = filteredRecipes.length;
  let error = "";

  counterWrapper.innerHTML = "";

  // Afficher le compteur de recettes
  count <= 1 ? counterWrapper.innerHTML = `<p>${count} recette</p>` : counterWrapper.innerHT

  if (count === 0) {
    // Afficher un message d'erreur s'il n'y a pas de recette correspondante à la recherche
    error = `<p>😧 Aucune recette ne contient la recherche ☞ "${searchBar.value.toLowerCase
    document.getElementById("error").innerHTML = error;
  } else {
    error = "";
    document.getElementById("error").innerHTML = error;
  }
}
```

LES UTILS

fetch_recipes.js

Ce code JavaScript gère efficacement une application de recettes en ligne. Il est composé de plusieurs listes et fonctions clés :

- `allRecipes` et `filteredRecipes` : Ces listes stockent toutes les recettes et les recettes filtrées respectivement.

- `ingredientList`, `utensilList`, et `applianceList` : Ce sont des listes pour stocker les ingrédients, les ustensiles et les appareils, respectivement.

- `tagList` : Une liste pour les tags.

- `searchValue` : Une variable pour stocker la valeur de recherche.

Le processus commence avec la fonction `init()` qui récupère les données des recettes depuis un fichier JSON en utilisant `getRecipes()`, puis initialise les listes et affiche la galerie de recettes. La fonction `updateList()` maintient les listes d'ingrédients, d'ustensiles et d'appareils à jour.

Ce code est essentiel pour gérer les données des recettes, les listes associées et la recherche, offrant une expérience utilisateur optimale sur le site web de recettes.

fetch_recipes

```javascript
let allRecipes = []; // Une liste contenant toutes les recettes
let filteredRecipes = []; // Une liste pour stocker les recettes filtrées

let ingredientList = []; // Une liste pour les ingrédients
let utensilList = []; // Une liste pour les ustensiles
let applianceList = []; // Une liste pour les appareils

let ingredientNewList = []; // Une nouvelle liste pour les ingrédients
let utensilNewList = []; // Une nouvelle liste pour les ustensiles
let applianceNewList = []; // Une nouvelle liste pour les appareils

let tagList = []; // Une liste pour les tags
let searchValue = ""; // La valeur de recherche


init();

// Récupère la liste des recettes
async function getRecipes() {
  try {
    const response = await fetch('./data/recipes.json'); // Récupère les données du fichier JSON
    const data = await response.json(); // Convertit les données au format JSON
    return data;
  } catch (error) {
    console.error(error);
  }
}

async function init() {
  const recipes = await getRecipes(); // Obtient les recettes
  allRecipes = recipes.recipes; // Stocke toutes les recettes dans la variable allRecipes
  filteredRecipes = allRecipes; // Initialise la liste des recettes filtrées avec toutes les recettes

  updateList(); // Met à jour les listes d'ingrédients, d'ustensiles et d'appareils
  displayGallery(allRecipes); // Affiche la galerie de recettes
  recipesCounter(filteredRecipes); // Met à jour le compteur de recettes
}
```

filterList.js

Ce code JavaScript gère la fonctionnalité de filtrage et d'affichage d'éléments sur une page web. Voici une brève description :

- Il utilise des ensembles (`ingredientSet`, `applianceSet`, `utensilSet`) pour stocker des éléments uniques, évitant ainsi les doublons.

- Le code prend en charge la sélection d'éléments dans des menus déroulants (`ingredientBtn`, `applianceBtn`, `utensilBtn`) et l'affichage des éléments sélectionnés.

- Il vérifie également les saisies de l'utilisateur dans les champs de recherche associés aux menus déroulants et filtre dynamiquement les éléments en fonction des saisies.

- Les éléments sélectionnés sont affichés sous forme de tags, et l'utilisateur peut les supprimer pour affiner sa recherche.

- La fonction `getFilteredRecipes()` filtre les recettes en fonction des éléments sélectionnés, de la recherche et de la présence de tags.

- La fonction `updateList()` met à jour les listes d'ingrédients, d'appareils et d'ustensiles en fonction des recettes filtrées.

Ce code offre une expérience utilisateur interactive pour explorer et filtrer des recettes en fonction d'ingrédients, d'appareils et d'ustensiles, avec une interface de recherche dynamique.

filterList.js

```javascript
// Créer des ensembles pour éviter les doublons.
let ingredientSet = new Set(); // Ensemble pour les ingrédients
let applianceSet = new Set(); // Ensemble pour les appareils
let utensilSet = new Set(); // Ensemble pour les ustensiles
let tempFilteredRecipes = []; // Liste temporaire des recettes filtrées

let selectedItem = []; // Liste des éléments sélectionnés

// Sélectionner les boutons du menu déroulant
const ingredientBtn = document.querySelector(".btn_ingredients"); // Bouton pour les ingrédients
const ingredientMenu = ingredientBtn.querySelector(".result_list"); // Menu des ingrédients
const applianceBtn = document.querySelector(".btn_appareils"); // Bouton pour les appareils
const applianceMenu = applianceBtn.querySelector(".result_list"); // Menu des appareils
const utensilBtn = document.querySelector(".btn_utensils"); // Bouton pour les ustensiles
const utensilMenu = utensilBtn.querySelector(".result_list"); // Menu des ustensiles

// Fonctions génériques
function checkListClick(list, menu) {
  const menuEl = menu.getElementsByTagName("li");

  // Parcourir tous les éléments <li> trouvés
  [...menuEl].forEach((li) => {
    // Écouter les clics sur les éléments
    li.addEventListener("click", () => {
      // Obtenir le texte de l'élément HTML
      let tag = li.textContent;

      // Créer le tag
      createTag(menu, tag);
    });
  });
}
```

searchbar.js

Ce code JavaScript gère la barre de recherche et les interactions associées sur une page web. Voici une brève description :

- Il récupère des éléments du DOM tels que la barre de recherche (`searchBar`), l'icône de loupe (`magnifyGlass`), l'icône de croix (`xMarkEl`), et écoute divers événements pour déclencher des actions.

- L'icône de loupe et la touche "Entrée" déclenchent la fonction `effectuerRecherche()` pour effectuer la recherche.

- La touche "Échap" déclenche la fonction `viderRecherche()` pour réinitialiser la barre de recherche.

- Le code réagit aux modifications dans le champ de texte de la barre de recherche et affiche l'icône de croix après que l'utilisateur ait entré au moins 3 caractères.

- La fonction `effectuerRecherche()` filtre les recettes si l'utilisateur a entré au moins 3 caractères, sinon elle réinitialise la barre de recherche.

- La fonction `viderRecherche()` réinitialise la barre de recherche, supprime l'icône de croix, et réaffiche les cartes de recettes.

Ce code offre une expérience de recherche conviviale, permettant aux utilisateurs de rechercher des recettes en entrant des mots-clés dans la barre de recherche, tout en fournissant des fonctionnalités de réinitialisation.

searchbar.js

```javascript
const searchBar = document.getElementById("search_input"); // L'élément de la barre de reche
const magnifyGlass = document.querySelector(".magnifyGlass_bg"); // L'icône de loupe
const xMarkEl = document.querySelector(".input_Xmark"); // L'icône de croix

// Écouteur de clic sur l'icône de loupe
magnifyGlass.addEventListener("click", () => {
  effectuerRecherche();
})

// Écouteur des touches "Entrée" et "Échap"
document.addEventListener("keydown", (e) => {
  if (e.key === "Enter") {
    e.preventDefault();
    effectuerRecherche();
  }
  if (e.key === "Escape") {
    e.preventDefault();
    viderRecherche();
  }
})

// Écouteur de clic sur la croix de la barre de recherche
xMarkEl.addEventListener("click", () => {
  viderRecherche();
})

// Écouteur des modifications dans le champ de texte. Affiche la croix après 3 caractères.
document.addEventListener("input", () => {
  if (searchBar.value.length !== 0) {
    xMarkEl.classList.add("XmarkShow");
  } else {
    xMarkEl.classList.remove("XmarkShow");      You, il y a 2 minutes • Uncommitted changes
  }
})
```

recipes.json

recipes.json

Ce fichier JSON contient des données de recettes de cuisine structurées en un tableau d'objets sous la clé "recipes".

Chaque objet représente une recette avec des propriétés telles que "id", "name", "servings", "ingredients", "time", "description", "appliance", et "ustensils".

Ces données fournissent des informations détaillées sur chaque recette, y compris les ingrédients, les étapes de préparation, et les équipements requis pour les réaliser.

recipes.json

```
{
    "recipes":  [
    {
        "id": 1,
        "name" : "Limonade de Coco",
        "servings" : 1,
        "ingredients": [
            {
                "ingredient" : "Lait de coco",
                "quantity" : 400,
                "unit" : "ml"
            },
            {
                "ingredient" : "Jus de citron",
                "quantity" : 2
            },
            {
                "ingredient" : "Crème de coco",
                "quantity" : 2,
                "unit" : "cuillères à soupe"
            },
            {
                "ingredient" : "Sucre",
                "quantity" : 30,
                "unit" : "grammes"
            },
            {
                "ingredient": "Glaçons"
            }
        ],
        "time": 10,
        "description": "Mettre les glaçons à votre goût dans le blender, ajouter le lait, la crème de coco,
        "appliance": "Blender",
        "ustensils": ["cuillère à Soupe", "verres", "presse citron" ]
```

Résumé

L'objectif principal est d'optimiser le moteur de recherche du site pour offrir une expérience fluide et performante aux utilisateurs.

Pour ce faire, le projet débute par la mise en place de la fonction de recherche, en se basant sur un fichier JavaScript contenant 50 recettes.

Le développement se déroule en trois étapes essentielles :

Conception de l'Interface

Dans un premier temps, l'interface utilisateur est mise en place, en respectant scrupuleusement la maquette conçue sur Figma.

Les technologies comme Bootstrap ou Tailwind CSS sont utilisées pour les styles, tout en garantissant la conformité avec les normes du validateur W3C.

Développement des Algorithmes

Ensuite, deux implémentations distinctes de la fonction de recherche sont élaborées.
L'une utilise des boucles natives (for, while), tandis que l'autre repose sur la programmation fonctionnelle avec les méthodes Array (forEach, filter, map, reduce).
Ces implémentations se concentrent sur la recherche principale, et des schémas (algorigrammes) sont élaborés pour bien décrire les étapes de chaque algorithme.

Comparaison des Performances

La performance des deux implémentations est évaluée en utilisant un outil de comparaison de performance tel que Jsben.ch.

Cela permet de mesurer le nombre d'opérations par seconde réalisées par chaque script et de déterminer quelle implémentation est la plus performante.

La recommandation finale pour l'algorithme à utiliser est ajoutée à la "fiche d'investigation de fonctionnalité".

Fin de la Soutenance Projet OpenClassrooms

Les petits plats

Développez un algorithme de recherche en JavaScript

Présenter par Ghani